

Lecture 3: Architectures and Hyperparameters

Recapping the Modern Transformer

Everything you didn't want to know about LM architecture and hyperparameters!

- What do most of the large language models (LLMs) have in common?
- What are common variations to the architecture / training process?

Theme: While the best way to learn is hands-on experience, the next best way is to try to learn from others' experience.

Google's "original" transformer (2017)

- **Positional embedding:** sines and cosines

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- **Feedforward Network (FFN):** ReLU

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- **Norm type:** post-norm, LayerNorm

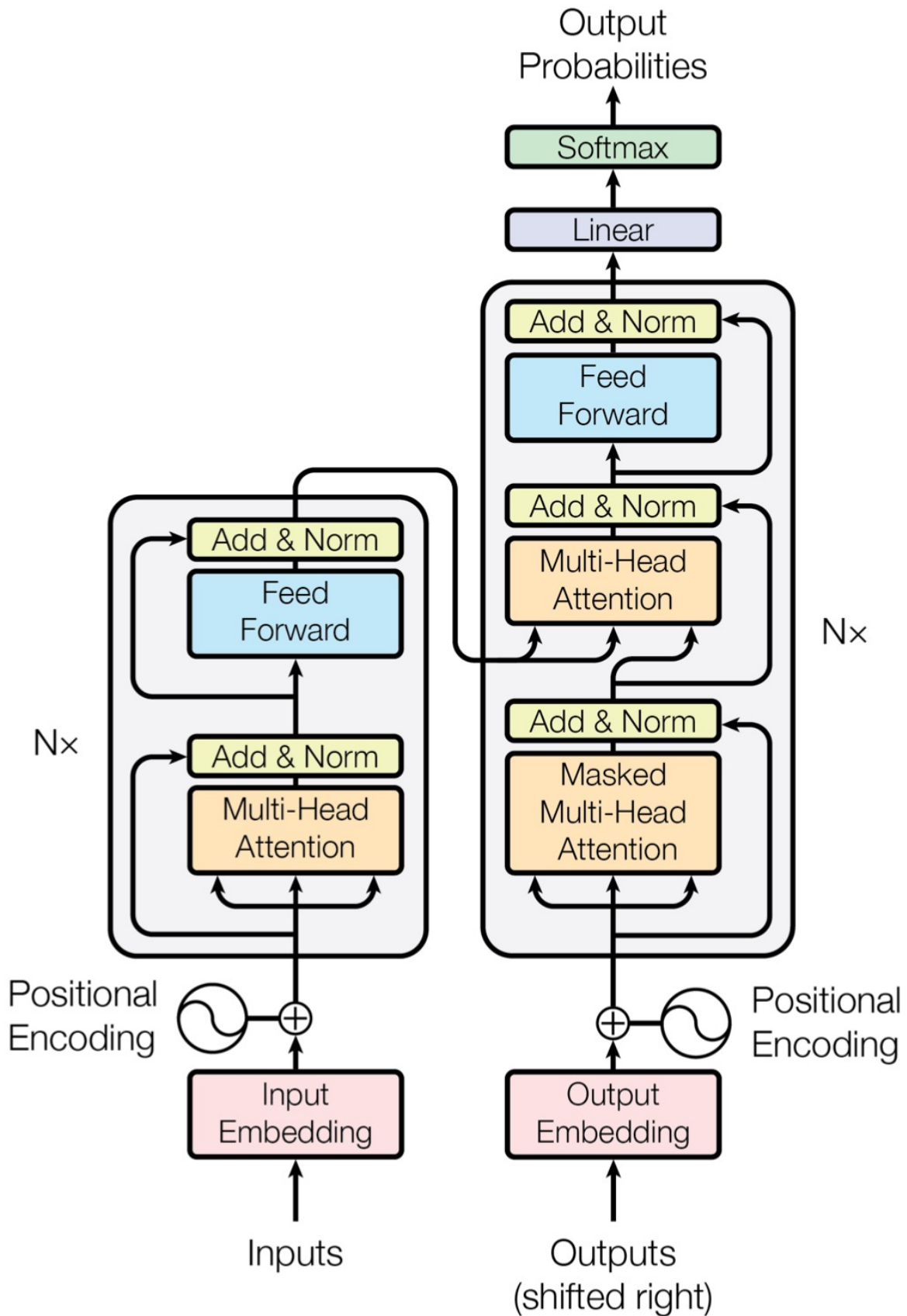
CS336's "modern" transformer (2026)

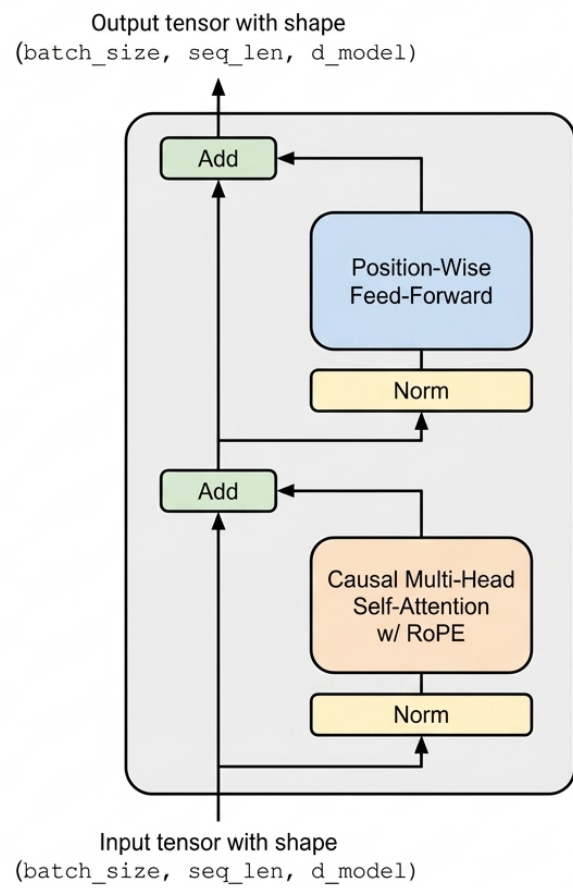
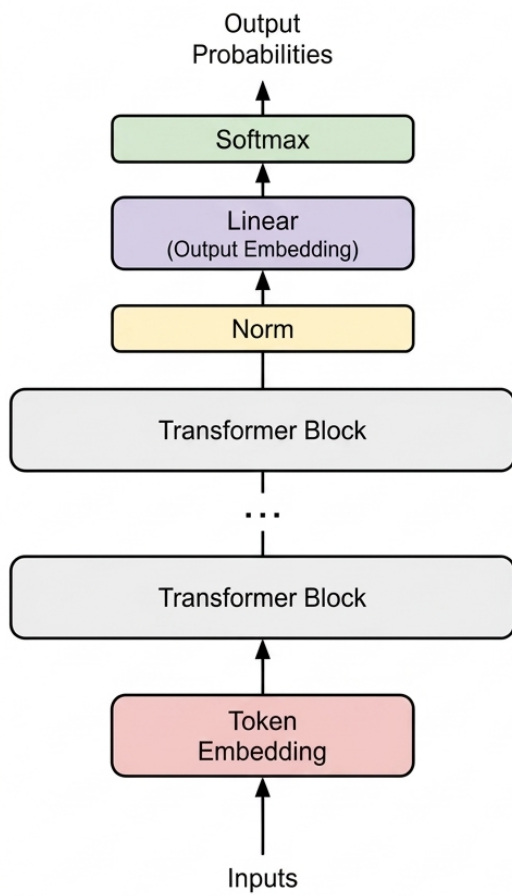
- **Positional embedding:** Rotary position embedding (RoPE)
- **FFN:** SwiGLU, not ReLU
- **Norm type:** pre-norm, LayerNorm
- **Affine operators:** Linear layers (and layernorm) have no bias (constant) terms

Questions: Why did we pick these? What should you pick?

Over 19 new *dense* model releases, many of them with minor architecture tweaks ...

- Let's look at the data (on dense architectures).
- Learn from the many other models (and papers) out there.





We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

What are we going to cover?

- Activations, feedforward networks
- Positional embeddings
- Hyperparameters that (do or don't) matter
- Stability tricks
- Attention variations

Questions: What is `ff_dim`? Do `multi_head_dims` always sum to `model_dim`? How many vocab elements?

Architecting the Modeling

Let's think about the core architecture piece:

- Dominance of "LLaMA-like" architectures.
- Trends over the years (QK-norm, Hybrid attention).

Pre-vs-Post Norm

The one thing that *everyone* agrees on (in 2024).

- **Almost all modern LMs use pre-norm (but BERT was post-norm).**
- (One somewhat funny exception - OPT350M. I don't know why this is post-norm).

Setup LayerNorm so that it doesn't affect the main residual signal path (on the left). Explanations?

- **Gradient propagation** - nicer attenuation and fewer spikes.
- **Original stated advantage** - removing the warmup.
- **Today** - stability and large learning rates (LRs) for large networks.

New things - "double" norm or non-residual post-norms.

- If putting LayerNorms in residual streams is bad ... Why not post-norm outside the stream?
- **Recent models:** Grok, Gemma 2. Olmo 2 *only* does non-residual post-norm.

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** - normalizes the mean and variance across d_{model} .

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\sigma^2[x] + \epsilon}} * \gamma + \beta$$

Notable models: GPT1/2/3, OPT, GPT-J, BLOOM.

Many modern LMs: **RMSNorm** - does not subtract mean or add a bias term.

$$y = \frac{x}{\sqrt{\|x\|^2 + \epsilon}} * \gamma$$

Notable models: LLaMA-family, PaLM, Chinchilla, T5.

Why RMSNorm?

Modern explanation - it's faster (and just as good).

- **Fewer operations** (no mean calculation).
- **Fewer parameters** (no bias term to store).

Does this explanation make sense? MatMuls are the *vast* majority of FLOPs (and memory).

Important lesson: FLOPS are not runtime (we will discuss this in far more detail later)!

RMSNorm can still matter due to the importance of *data movement* and runtime (plus surprisingly, performance) gains have been seen in papers.

More generally: dropping the bias terms

Remark: *Most modern transformers don't have bias terms.*

Most implementations (if they're not gated):

$$FFN(x) = \sigma(xW_1)W_2.$$

Reason: memory (similar to RMSNorm) and optimization stability.

Activating the Neurons

A whole zoo of activation functions ..

ReLu, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, SeLU, SwiGLU, LiGLU

What are these things? What do people use? Does it matter?

Common Activations

- $ReLU(x) = \max(0, x) \implies FFN(x) = \max(0, xW_1)W_2$.
- **Notable models:** Original transformer, T5, Gopher, Chinchilla, OPT.
- $GeLU(x) = x\Phi(x) \implies FFN(x) = xW_1\Phi(xW_1)W_2$.
- **Notable models:** GPT1/2/3, GPT-J, GPT-Neox, BLOOM.

Gated Activations (*GLU)

GLUs modify the "first part" of a FF layer.

$$FFN(x) = \max(0, xW_1)W_2$$

Instead of linear + ReLU, augment the above with an (entrywise) linear term.

$$\max(0, xW_1) \rightarrow \max(0, xW_1) \otimes (xV)$$

This gives the gated variant (ReGLU) - note that we have an extra parameter (V).

$$FFN_{ReGLU}(x) = (\max(0, xW_1) \otimes xV)W_2$$

Gated Variants of Activations

- $FFN_{GeGLU}(x, W_1, V, W_2) = (GeLU(xW_1) \otimes xV)W_2$.
- **Notable models:** T5 v1.1, mT5, LaMDA, Phi3, Gemma 2/3/4.
- $FFN_{SwiGLU}(x, W_1, V, W_2) = (Swish(xW_1) \otimes xV)W_2$.
- **Notable models:** LLaMa 1/2/3, PaLM, Mistral, OLMo, *most models post 2023*.
- Note: Gated models use smaller dimensions for the d_{ff} by 2/3.

Do gated linear units work?

- Yes, fairly consistently so.
- Yes, with other works corroborating Shazeer 2020.

Many variations (ReLU, GeLU, *GLU) across models.

- *GLU isn't *necessary* for a working model (see GPT3), but it's rare to see others .. some outlier models .. Nemotron 340B (Squared ReLU).
- Evidence points towards somewhat consistent gains from Swi / GeGLU.

Serial vs Parallel Layers

Normal transformer blocks are *serial* - they compute attention, then the MLP.

Question: *Could we parallelize the transformer block?*

A few models (GPTJ, PaLM, GPT-NeoX) do parallel layers. If implemented right, LayerNorm can be shared and matrix multiples can be fused.

Claim (Recent Models): *Cohere Command A, Falcon 2 11B, Command R+.*

Most models now use serial layers.

Positioning the Embeddings

Many variations in positional embeddings ...

- **Sine embeddings:** adds sines and cosines that enable localization

$$Embed(x, i) = v_x + PE_{pos}$$

- **Notable models:** original transformer.
- **Absolute embeddings:** add a position vector to the embedding

$$Embed(x, i) = v_x + u_x$$

- **Notable models:** GPT1/2/3, OPT.
- **Relative embeddings:** adds a vector to the *attention computation*

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

- **Notable models:** T5, Gopher, Chinchilla.

RoPE: rotary position embeddings

High level thought process: a *relative* position embedding should be $f(x, i)$ such that

$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j).$$

That is, the attention function *only* gets to depend on the relative position $(i - j)$.

Question: *How do existing embeddings not fulfill this goal?*

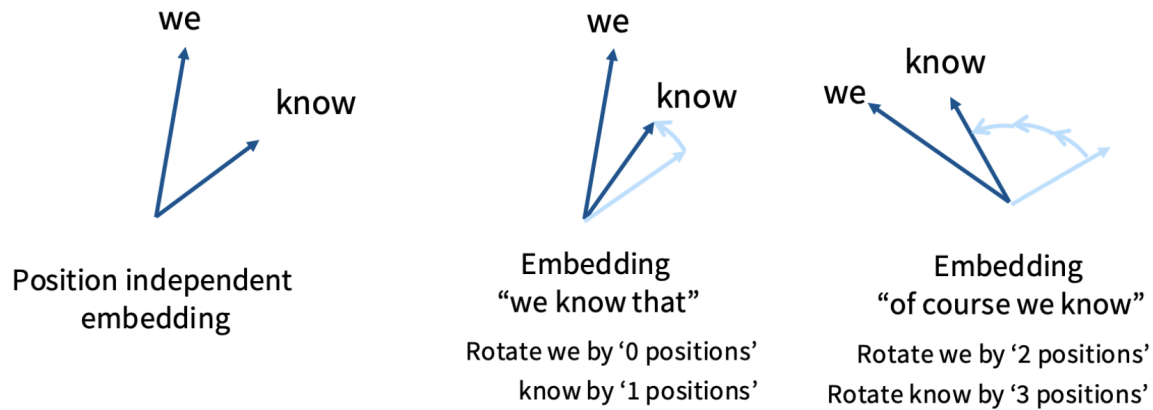
- **Sine:** Has various cross-terms that are not relative

$$\langle \text{Embed}(x, i), \text{Embed}(y, i) \rangle = \langle v_x, v_y \rangle + \langle PE_i, v_y \rangle \dots$$

- **Absolute:** obviously not relative
- **Relative embeddings:** e_{ij} is NOT an inner product

How can we solve this problem?

- We want our embeddings to be invariant to absolute position.
- We know that inner products are invariant to arbitrary rotation.



There are many rotations, which one do you pick? Just pair up coordinates and rotate them in 2d (motivation: complex numbers).

The RoPE math

What is the inner product $f_{\{q,k\}}$?

$$f_{\{q,k\}}(x_m, m) = R_{\theta,m}^d W_{\{q,k\}} x_m$$

What is the rotation matrix $R_{\theta,m}^d$?

$$R_{\theta,m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{bmatrix}$$

Difference with sine embeddings - not additive, no cross terms.

Code for RoPE

```

query_states = self.q_proj(hidden_states)
key_states = self.k_proj(hidden_states)
value_states = self.v_proj(hidden_states)

# Flash attention requires the input to have the shape
# batch_size x seq_length x head_dim x hidden_dim
# therefore we just need to keep the original shape
query_states = query_states.view(bsz, q_len,
    self.num_heads, self.head_dim).transpose(1, 2)
key_states = key_states.view(bsz, q_len,
    self.num_key_value_heads, self.head_dim).transpose(1, 2)
value_states = value_states.view(bsz, q_len,
    self.num_key_value_heads, self.head_dim).transpose(1, 2)

# Get the RoPE matrix cos/sin
cos, sin = self.rotary_emb(value_states, position_ids)

# Multiply query/key inputs
query_states, key_states = apply_rotary_pos_emb(
    query_states, key_states, cos, sin)

```

Note: embedding at *each attention operation* to enforce position invariance.

Engineering the Hyperparameters**Transformer hyperparameter questions you might have had in CS224N ..**

- How much bigger should the feedforward size be compared to hidden size?
- How many heads and should num_heads always divide hidden size?
- What should my vocab size be?

And other model setting questions

- Do people even regularize these huge LMs?
- How do people scale these models - very deep or very wide?

Surprising (?) consensus hyperparameter 1

Feedforward - model dimension ratio.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant - the feedforward dim (d_{ff} and model dim (d_{model}). What should their relationship be?

$$d_{ff} = 4d_{model}$$

This is *almost always* true. There's just a few exceptions.

Exception #1 - GLU variants

Remember that GLU variants scale down by $2/3$. This means most GLU variants have

$$d_{ff} = \frac{8}{3}d_{model}.$$

Models are roughly in this range, though PaLM, LLaMA2 and Mistral are slightly larger.

Exception #2 - T5

As we have (and will) see, most LMs are have conservative hyperparameters. One exception is T5 [Raffel et al 2020] which has some *very bold* settings. In particular, for the 11B model, the T5 team set

$$\begin{aligned} d_{ff} &= 65,536 \\ d_{model} &= 1024 \end{aligned}$$

For an astounding 64-times multiplier.

We chose to scale up d_{ff} specifically because modern accelerators (such as the TPUs we train our models on) are most efficient for large dense matrix multiplications like those in the Transformer's feed-forward network. - Raffel et al 2020

Other, recent exceptions - Gemma 2 ($8\times$), SmolLM / Gemma 3 / Gemma 4 ($4\times$, GLU).

Question: *Why this range of multiplier?*

Empirically, there's a basin between 1-10 where this hyperparameter is near-optimal.

Question: *What can we learn from the model-dim hyperparameter?*

- The "default" choices of $d_{ff} = 4d_{model}$ and $d_{ff} = 2.66d_{model}$ have worked well for nearly all modern LLMs.
- But T5 does show that even radical choices of $d_{ff} = 64d_{model}$ can work. This hyperparameter choice isn't written in stone.
- That said, T5 has a follow-up model (T5 v1.1) that is "improved" and uses a much more standard 2.5 multiplier on GeGLU, so the 64-times multiplier is likely suboptimal.

Suprising (?) consensus hyperparameter 2

Even though we compute h many attention heads, it's not really more costly.

- We compute $XQ \in \mathbb{R}^{n \times d}$ and then reshape to $\mathbb{R}^{n \times h \times d/h}$ (likewise for XK and XV).
- Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
- Almost everything else is identical and **the matrices are the same sizes**.

Claim: $\text{num-heads} \times \text{head-dimension} = \text{model-dimension}$.

This doesn't *have to* be true: we can have head-dimension $>$ model-dimension / num-heads. But most models do follow this guideline.

Remark: *Most models have around 1 - notable exceptions by some Google models.*

Considerations about aspect ratios

Should my model be deep or wide? *How deep and how wide?*

$$\frac{d_{\text{model}}}{n_{\text{layer}}} \approx 100.$$

Question: *Most models are surprisingly consistent on this one too! Sweet spot?*

Extremely deep models are harder to parallelize and have higher latency.

We note an obvious limitation with our advice. Scaling depth has an obvious limiter, i.e, they are non-parallelization across different machines or devices and every computation has to always wait for the previous layer. This is unlike width, which can be easily parallelizable over thousands or hundreds of thousands of devices. - Tay et al 2021

Question: *What are typical vocabulary sizes?*

- Monolingual models: 30-50k words.
- Multilingual / production systems: 100-250k words.

Monolingual vocabs don't need to be huge, but multilingual ones do.

Dropout and other regularization

Question: *Do we need regularization during pretraining?*

Augments against:

- There is *a lot* of data (trillions of tokens), more than parameters.
- SGD only does a single pass on a corpus (hard to memorize).

This is all quite reasonable .. but what do people do in practice?

- Many older models used dropout during pretraining.
- Newer models (except Qwen) rely only on weight decay.

Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout.

Question: *Why weight decay LLMs?*

[Andriushchenko et al 2023] has interesting observations about LLM weight decay.

- It's not to control overfitting.
- Weight decay interacts with learning rates (cosine schedule).
- Its effects are primarily on optimization dynamics.

Stability Tricks

Recently, lots of attention on *stable training*.

Where do the issues arise? Beware of softmaxes!

Softmaxes - can be ill-behaved due to exponentials / divisions by zero.

$$\begin{aligned}\log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \sum_{r'=1}^{|V|} e^{U_{r'}(x)}\end{aligned}$$

Output softmax stability - "z-loss" trick (Devlin 2014)

$$\begin{aligned}
L &= \sum_i [\log(P(x_i) - \alpha(\log(Z(x_i)) - 0)^2] \\
&= \sum_i [\log(P(x_i)) - \alpha \log^2(Z(x_i))]
\end{aligned}$$

This is useful for stability! PaLM used this "z-loss" trick.

We additionally use an auxiliary loss of $\Delta z = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log Z$ to be closer to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2/3 (2025).

Attention softmax stability - the "QK norm" (Dehgani 2023, Idefcs, Chameleon)

- The query (Q) and keys (K) are layer (RMS) normed before going into softmax operation.
- **Other examples:** DCLM, OLMo2, Gemma 2, Qwen3, OLMo 3, Gemma 4.

Logit soft-capping

Soft-capping the logits to some maximum value via $\tanh$.

We cap logits (Bello et al 2016) in each attention layer and the final layer such that the value of the logits stays between $-\text{soft_cap}$ and $+\text{soft_cap}$. More specifically, we cap the logits with the following function:

$$\text{logits} \leftarrow \text{softcap} * \tanh(\text{logits} / \text{softcap}).$$

Prevents logits from blowing up, but also might have performance issues?

Attending to Heads

Most models don't touch the attention heads much at all with a few minor exceptions ..

- **GQA / MQA:** Saving inference costs by reducing the number of heads.
- **Sparse or sliding window attention (GPT4 / Mistral):** restricting the attention pattern to reduce compute cost.
- **Exotic SSM stuff (Jamba, Falcon 3, Qwen 3.5, etc):** next lecture!

Reducing attention head cost

Let's think about the compute involved for attention.

$$\text{Atten}(X, Q, K, V) = \rho(XQK^T X^T)XV$$

Total arithmetic operations (bnd^2) and **total memory accesses** ($bnd + bhn^2 + d^2$).

Arithmetic intensity (compute / memory) is high $O\left(\left(\frac{1}{k} + \frac{1}{bn}\right)^{-1}\right)$ - we can keep our GPUs running.

Question: What about the incremental case when we generate text?

Key difference: can't parallelize the generation process - needs to be step by step.

Remark: In this case - we need to incrementally re-compute/update attention via the "KV cache".

Question: What's the incremental arithmetic intensity?

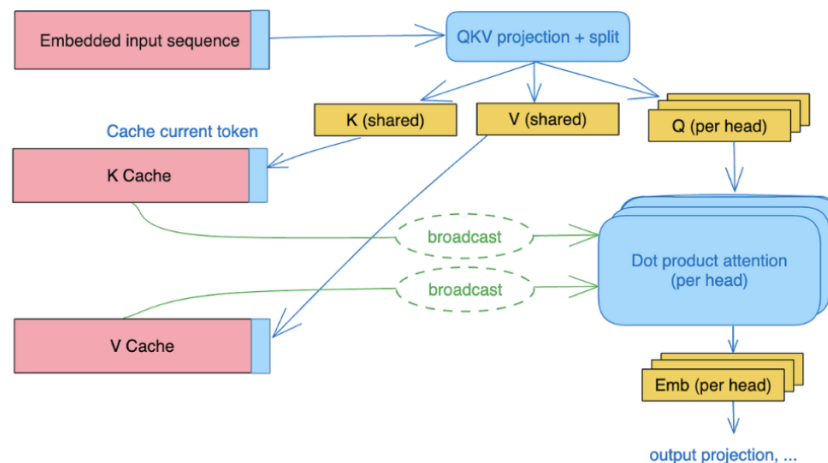
Total arithmetic operations (bnd^2) and **total memory accesses** ($bn^2d + nd^2$).

Arithmetic intensity is low $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - we need large batches + short sequences or big model dimensions.

Question: Is there some way around this? The n/d term is difficult to reduce.

MQA - just have fewer key dimensions

Key idea - have multiple queries, but just one dimension for keys and values.

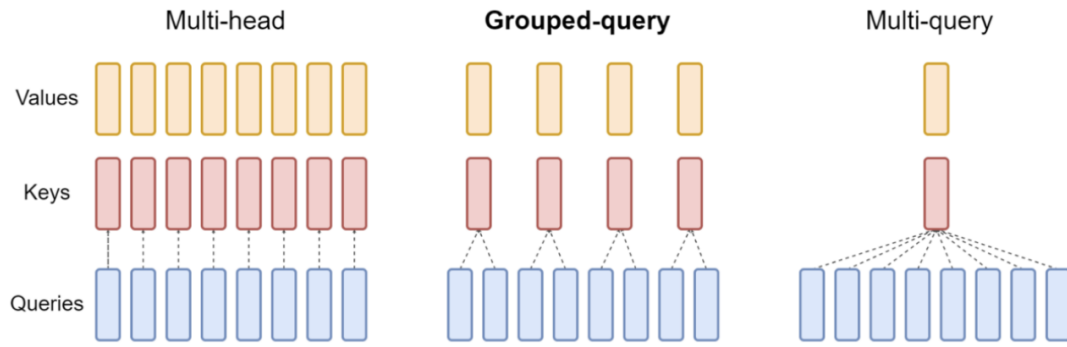


Claim: We have much fewer items to move in and out of memory (KV cache).

Memory accesses ($bnd + bn^2k + nd^2$) and **arithmetic intensity** ($O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$).

GQA - additional extensions

Don't do all the way to one dimension of KV - have fewer dimensions.

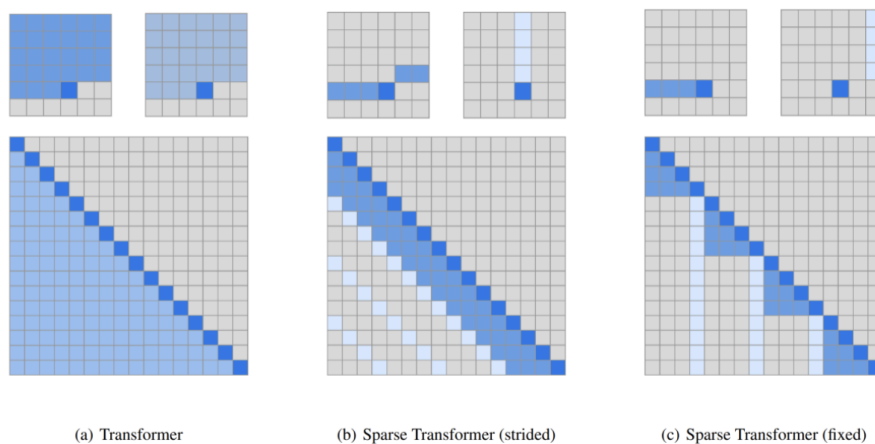


Claim: Simple knob to control expressiveness (key-query ratio) and inference efficiency.

More recently - MLA (multihead latent attention) from DeepSeek v2.

Question: Does MQA / GQA hurt? Sometimes ..

- Small hit with MQA [Shazeer 2019].
- Low hit with GQA [Ainslie 2023].

Sparse / sliding window attention

Remark: Attending to the entire context can be expansive (quadratic).

Build sparse / structured attention that trades-off expressiveness verse runtime (GPT3, GPT-OSS, Gemma 4).

